

Programmation Fonctionnelle

TD n° 4 : Arbres

1 Fonctions de base sur les arbres

On considère le type OCaml suivant :

```
type 'a tree = Nil | Node of 'a * 'a tree * 'a tree
```

Écrire les fonctions suivantes.

Exercice 1 : size : `'a tree -> int` renvoyant la taille d'un arbre, c'est-à-dire son nombre de nœuds.

Exercice 2 : depth : `'a tree -> int` renvoyant la profondeur d'un arbre, c'est-à-dire la longueur de sa plus longue branche. Par exemple, l'arbre vide est de profondeur 0, un arbre réduit à une feuille (un seul nœud, sans fils gauche ni fils droit) est de profondeur 1, etc.

Exercice 3 : sum : `int tree -> int`, qui calcule la somme des étiquettes d'un arbre.

Exercice 4 : contains : `'a -> 'a tree -> bool`, telle que l'application `contains x a` calcule `true` si et seulement si l'un des nœuds de l'arbre `a` est étiqueté par `x`.

Exercice 5 : elements : `'a tree -> 'a list` qui calcule la liste des éléments présents dans l'arbre en les collectant de la gauche vers la droite (étiquettes du sous-arbre gauche, étiquette à la racine, étiquettes du sous-arbre droit). Sauriez-vous écrire une version sans utiliser la concaténation (`@`) des listes ?

Exercice 6 : perfect : `'a tree -> bool` déterminant si un arbre est parfait, c'est-à-dire si toutes ses feuilles sont à la même profondeur. Noter que tester cette propriété sur les sous-arbres gauche et droit ne suffit pas (pourquoi ?). Sauriez-vous écrire une version avec un seul parcours de l'arbre ?

2 Arbres binaires de recherche

Rappelons qu'un **arbre binaire de recherche** (BST pour "Binary Search Tree") est un arbre binaire étiqueté par les entiers vérifiant à chaque nœud la propriété suivante : si la racine de ce nœud est étiquetée par `n` alors, (1) toutes les étiquettes de son fils gauche sont strictement inférieures à `n`, (2) toutes les étiquettes de son fils droit sont strictement supérieures à `n`.

Exercice 7 : contains_bst : `int -> int tree -> bool` telle que si `a` est un BST, l'application `contains_bst x a` calcule `true` si et seulement si l'un des nœuds de l'arbre `a` est étiqueté par `x`. L'exploration de `a` devra être minimale, en tenant compte du fait qu'il s'agit d'un BST.

Exercice 8 : add_bst : `int -> int tree -> int tree` telle que si `x` est un entier et `a` un BST, `add_bst x a` calcule l'arbre `a` dans lequel on a ajouté `x` s'il n'y est pas déjà – si `x` apparaît déjà dans `a`, le résultat est `a`. Le résultat doit être encore un BST, et doit être une extension de `a`.

Exercice 9 : `bst_of_list : int list -> int tree` construisant à partir d'une liste d'entiers un BST contenant les mêmes éléments. Cette fonction peut utiliser la précédente. Vérifier sur des exemples que `elements (bst_of_list l)` et `l` contiennent bien les mêmes éléments.

Quelle forme de BST obtient-on lorsque la liste est triée? Est-ce un problème?

Exercice 10 : `bst_of_list_opt : int list -> int tree` transformant une liste triée et sans répétitions d'entiers en un BST contenant les mêmes éléments, et de profondeur minimale.

Vérifier sur des exemples que `elements (bst_of_list_opt l) = l`. Pour limiter la profondeur de l'arbre obtenu, on pourra écrire une fonction auxiliaire découpant une liste en deux moitiés.

Exercice 11 : (plus difficile) `is_bst : int tree -> bool` prenant en argument un arbre quelconque, et déterminant s'il s'agit d'un BST. Il pourra être utile de définir plusieurs fonctions auxiliaires.

3 Fonctions d'ordre supérieur

Exercice 12 : Sans déclarer de fonctions auxiliaires et sans vous servir des fonctions prédéfinies, écrire : `forall_labels : ('a -> bool) -> 'a tree -> bool` telle que `forall_labels p a` calcule `true` si et seulement si chaque étiquette `x` de `a` vérifie `p x = true`.

Exercice 13 : En utilisant la fonction `forall_labels` de l'exercice précédent, écrire une fonction `is_uniform : 'a -> 'a tree -> bool` telle que `is_uniform v a` calcule `true` si et seulement si chaque étiquette de `a` est égale à `v`. Cette fonction ne devra pas être déclarée comme récursive, mais doit déléguer entièrement la récurrence à la précédente.

Exercice 14 : Toujours sans fonctions auxiliaires ou prédéfinies, écrire une fonction `forall_subtrees : ('a -> 'a tree -> 'a tree -> bool) -> 'a tree -> bool` telle que `forall_subtrees p a` calcule `true` si et seulement si pour chaque sous-arbre de `a` de la forme `Node(x, g, d)`, on a `p x g d = true`.

Exercice 15 : On dit qu'un arbre est un *peigne droit* s'il est réduit à l'arbre vide, ou si son fils gauche est vide et son fils droit est un peigne droit. À partir de la seule fonction `forall_subtrees`, écrire la fonction `is_right_comb : 'a tree -> bool`, telle que `is_right_comb a` calcule `true` si et seulement si `a` est un peigne droit, et déléguant entièrement la récurrence à la fonction `forall_subtrees`.

Exercice 16 : On définit pour les arbres un itérateur de la manière suivante :

```
let rec fold_tree fn vf a =
  match a with
  | Nil -> vf
  | Node(n, g, d) -> fn n (fold_tree fn vf g) (fold_tree fn vf d) ;;
```

Noter que `vf` spécifie la valeur à renvoyer si l'arbre `a` est réduit à un arbre vide. Si `a` est non vide, l'itérateur est appliqué à ses fils avec les mêmes arguments, et les valeurs de retour sont combinées à la valeur étiquetant la racine via la fonction `fn`. Quel est le type de `fold_tree`?

En utilisant uniquement `fold_tree`, et en lui déléguant la récurrence, écrire une fonction `sum : int tree -> int` qui calcule la somme des étiquettes de l'arbre que on lui passe comme argument.

Exercice 17 : Avec les mêmes contraintes, écrire :

`map_tree : ('a -> 'b) -> 'a tree -> 'b tree`

telle que `map_tree f a` calcule l'arbre obtenu en remplaçant chaque étiquette `x` de `a` par `(f x)`.